



Transforming Computing Education through GitHub and Version Control

Dr. Sheena N.
Assistant Professor
Dept. of Computer Science and Engineering
CAPE College of Engineering Alappuzha

Dr. Sheena N.

Overview

01	02
Why Move to Git and GitHub	Version Control Systems
03	04
Getting Started with Git	Working with GIT and GitHub
05	06
Branching and merging with GitHub	Collaboration using GitHub
07	
Transforming Computing Education	

Dr. Sheena N.

Why Move to Git and GitHub?

Computing Education Needs:

 Continuous assessment	 Collaborative learning
 Industry alignment	 Project-based education
 Academic integrity	 Digital portfolios
 Better assignment management	 Enhanced student engagement

Git and GitHub transform programming education from merely submitting code to continuously developing, collaborating, and learning through code.

Dr. Sheena N.

Traditional Education vs GitHub-Based Education

Aspect	Traditional Method	Git & GitHub
Assignment Submission	Email/Pen drive	Repository
Version Tracking	Not Available	Complete History
Collaboration	Difficult	Easy
Backup	Limited	Cloud-based
Assessment	Final Submission Only	Continuous Assessment
Team Projects	Difficult	Simple
Industry Exposure	Limited	High
Portfolio Creation	Not Available	Available

Dr. Sheena N.

Version Control Systems

A Version Control System (VCS) is a system that records changes to files over time so that specific versions can be recalled later and multiple users can collaborate efficiently on the same project.

Key Features

- Maintains a complete history of changes.
- Tracks who made each change and when.
- Allows restoration of previous versions.
- Supports collaboration among multiple users.
- Prevents accidental loss of work.
- Facilitates branching and merging of code.

Dr. Sheena N.

Types of Version Control Systems (VCS)

Type of VCS	Description	Architecture	Examples
Local Version Control System (LVCS)	Stores all versions on a single computer. Suitable for individual work.	User → Local Repository	RCS (Revision Control System)
Centralized Version Control System (CVCS)	A central server stores all versions, and users access the repository through the server.	Users → Central Server	CVS(Concurrent version System), Subversion (SVN)
Distributed Version Control System (DVCS)	Every user has a complete copy of the repository and its history. Supports offline work and collaboration.	User ↔ Repository ↔ User	Git, Mercurial

LVCS → Single Computer | CVCS → Central Server | DVCS → Complete Repository on Every System

Dr. Sheena N.

GIT and GitHub

GIT

- A distributed version control system
- Git works on your computer, but you also use it with online services like **GitHub**, **GitLab**, or **Bitbucket** to share your work with others. These are called **remote repositories**.

GitHub

- A web-based platform that hosts Git repositories and enables collaboration through pull requests, issues and team management.
- Hosted on the cloud (web platform)

Working with GIT

Download Git for free from git-scm.com.

1	2
Windows Download and run the installer. Click "Next" to accept the recommended settings. This will install Git and Git Bash.	Linux Open your terminal and use your package manager. For example, on Ubuntu: <code>sudo apt-get install git</code>

Git Bash

- Git Bash is a terminal for Windows that lets you use Git commands.
- After installing Git, you can find Git Bash in your Start menu.
- You can use Git Bash just like the Command Prompt, but with extra Unix commands (like `ls` and `pwd`).

Working with GIT

Check Git Version

```
git --version
```

Configure Git

Now let Git know who you are. This is important for version control systems, as each Git commit uses this information:

User Name

Your name will be attached to your commits. Set it with:

```
git config --global user.name "Your Name"
```

Email Address

Your email is also attached to your commits. Set it with:

Example

```
git config --global user.email "you@example.com"
```

Scope Options

Use `--global` to set the value for **every repository** on your computer. Use `--local` (the default) to set it only for the current repository.

Viewing Your Configuration

You can see all your Git settings with:

Example: List All Settings

```
git config --list
```

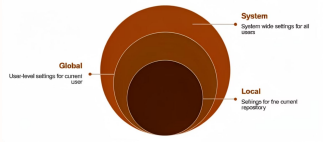
Working with GIT

Configuration Levels

There are three levels of configuration:

System (all users)	Global (current user)	Local (current repo)
<code>git config --system</code>	<code>git config --global</code>	<code>git config --local</code>

The order of precedence is:



Working with GIT

Key Steps to Get Started

- Create a project folder
- Navigate to the folder
- Initialize a Git repository

```
git init
```

You just created your first Git repository!

What is a repository?

A Git **repository** is a folder that Git tracks for changes. The repository stores all your project's history and versions.

Working with GIT

What is the Staging Environment?

The **staging environment** (or **staging area**) is like a waiting room for your changes. It tell Git exactly which files you want to include in your next commit.

This gives you control over what goes into your project history.

Here are some key commands for staging:

git add <file> Stage a file	git add --all or git add -A Stage all changes
git status See what is staged	git restore --staged <file> Unstage a file

Working with GIT

What is a Commit?

A **commit** is like a save point in your project. It records a snapshot of your files at a certain time, with a message describing what changed. You can always go back to a previous commit if you need to.

Here are some key commands for commits:

<code>git commit -m "message"</code> Commit staged changes with a message	<code>git commit -a -m "message"</code> Commit all tracked changes (skip staging)	<code>git log</code> See commit history
--	--	--

Push to GitHub

`git push <filename>`

Dr. Sheena N.

Branching and Merging in GitHub

Creating a New Branch

We are working in our local repository, and we do not want to disturb or possibly wreck the main project.

So we create a new branch:

`git branch <branch-name>`

Listing All Branches

To see all branches in your repository, use:

`git branch`

Dr. Sheena N.

Branching and Merging in GitHub

Switching Between Branches

`checkout` is the command used to check out a branch. Moving us **from** the current branch to the one specified at the end of the command:
`git checkout <branch-name>`

Dr. Sheena N.

Branching and Merging in GitHub

What is Merging in Git?

Merging in Git means combining the changes from one branch into another. This is how you bring your work together after working separately on different features or bug fixes.

Common git merge Options

<code>git merge</code> Merge a branch into your current branch	<code>git merge --no-ff</code> Always create a merge commit
<code>git merge --squash</code> Combine changes into a single commit	<code>git merge --abort</code> Abort a merge in progress

Dr. Sheena N.

Branching and Merging in GitHub

Merging Branches (git merge)

To combine the changes from one branch into another, use `git merge`. Usually, you first switch to the branch you want to merge *into* (often main or master), then run the merge command with the branch name you want to combine in.

First, we need to change to the master branch:

`git checkout master`

Now we merge the current branch (master) with emergency-fix:

`git merge emergency-fix`

Dr. Sheena N.

Cloning a Repository

A clone is a full copy of a repository, including all logging and versions of files.

Move back to the **original** repository, and click the green "Code" button to get the URL to clone: Open your Git Bash and clone the repository:

`git clone URL`

Dr. Sheena N.

Pulling a Branch from GitHub

Lets pull from our GitHub repository again so that our code is up-to-date:

01	02
Step 1	Step 2
Clone the repository or update your local repository with the latest changes.	Switch to the base branch of the pull request.
<code>git pull origin main</code>	<code>git checkout main</code>
03	04
Step 3	Step 4
Merge the head branch into the base branch.	Push the changes.
<code>git merge <head-branch></code>	<code>git push -u origin main</code>

Dr. Sheena N.

Git pull from GitHub.

Fetch, Pull and Merge

When working as a team on a project, it is important that everyone stays up to date. Any time you start working on a project, you should get the most recent changes to your local copy.

With Git, you can do that with `pull`.

`pull` is a combination of 2 different commands: `fetch` and `merge`

How fetch, merge, and pull work.

`git fetch` downloads new data from a remote repository but does not change your working files or branches. It lets you see what others have pushed before you merge or pull.

`git fetch origin`

To know recent changes, we can check our status:

`git status`

To view the log

`git log origin/master`

showing the differences between our local master and origin/master:

`git diff origin/master`

Now we can safely merge.

Dr. Sheena N.

Git Push Branch to GitHub

Push Branch to GitHub

How to push a branch from your local computer to GitHub. Create a new local branch, make a change, and push it to GitHub.

<code>git checkout -b update-readme</code>
Switched to a new branch "update-readme"
Edit a file, then check the status:
<code>git status</code>
Add and commit your changes:
<code>git add README.md</code>
<code>git commit -a "update readme for GitHub"</code>
Push your branch to GitHub:
<code>git push origin update-readme</code>

Dr. Sheena N.

Collaborate on GitHub

Create the repository

01	02
Go to http://github.com → + top right → New repository	Name: your project name
03	04
Choose Public or Private	Tick Add a README file → Create repository
Add team members	
01	02
In the repo, click Settings tab → Collaborators in left menu	Click Add people → Enter their GitHub usernames/emails
03	04
Give Write access	They'll get email invite → they must accept

Dr. Sheena N.

Protect the main branch if needed

01	02	03
Settings → Branches → Add branch protection rule	Branch name pattern: <code>main</code>	Tick: Require a pull request before merging
04	05	
Tick: Require approvals → Set 1 approval minimum	Save changes. Now no one can push directly to <code>main</code> .	

Dr. Sheena N.

Workflow for the Collaborators

Step 1: Clone the repo once

<code>git clone https://github.com/798-username/lotus-bloom-app.git</code>
<code>cd lotus-bloom-app</code>

Step 2: Create your own branch for every task

Never work on `main`. Always:

<code>git checkout main</code>
<code>git pull origin main</code>
<code>git checkout -b # branch name</code>

Step 3: Do work + commit

<code>git add .</code>
<code>git commit -m "message"</code>

Dr. Sheena N.

Step 4: Push your branch

```
git push origin branchname
```

Step 5: Create Pull Request (PR)

01	02	03
GitHub will show green banner Compare & pull request → click it	Base: main ← Compare: branch-name/filename	Give title and description
04	05	
Right side Reviewers → select Reviewer	Create pull request	

Dr. Sheena N.

Step 6: Code review & merge

01	02
PM/other member opens PR → Files changed tab → review code	Click Review changes → Approve or Request changes
03	04
If approved, PM clicks Merge pull request → Confirm merge	Click Delete branch to clean up

Step 7: Update your local main

After merge, everyone runs:

```
git checkout main
```

```
git pull origin main
```

Now you have everyone's latest code.

Dr. Sheena N.

GitHub Classroom

PART 1 : FACULTY SIDE

Step 1: Create Organization + Classroom

- Go to [github.com](#) → Top right + → New organization
Org name: **ML-Lab-VCET-2026**
- Go to [classroom.github.com](#) → Sign in → New classroom
Classroom name: **ML-Lab-4thSem-2026**
Organization: **ML-Lab-VCET-2026**

Step 2: Create Template Repo with 5 Experiments

This is the master copy students will get.
Create repo **ml-lab-template** under org **ML-Lab-VCET-2026**. Tick **README**.
Add folder structure:

Update main README.md

Name: [Student ID] | Roll No: [Student ID]

Experiment List

No	Title	Notebook	PDF	Status	Faculty Sign
1	Linear Regression	Lab01	PDF1	Pending	

☒ **Submission Rule:** Commit + Push before leaving lab. Export PDF from notebook with outputs.

Dr. Sheena N.

GitHub Classroom

PART 1 : FACULTY SIDE

Step 3: Create Assignment + Get Link

- Classroom → New assignment
Title: **ML-Lab-2026-5-Experiments**
Deadline: add deadline
Individual assignment: **Yes**
Repository prefix: **ml-lab-2026-**
Template repository: **ML-Lab-VCET-2026/ml-lab-template**
Grant students admin access: **No**
Enable feedback pull requests: **Yes**
- Click Create assignment → Copy invitation link:
<https://classroom.github.com/s/abc123xyz>
- Share this 1 link with all students

Dr. Sheena N.

GitHub Classroom

PART 2 : STUDENT SIDE

Step 1: One-time Join

- Click faculty's invitation link <https://classroom.github.com/s/abc123xyz>
- Authorize GitHub → Accept this assignment
- GitHub shows "You're ready to go!" You accepted the assignment. ML-lab-2026-5-Experiments. Your assignment repository has been created: <https://github.com/ML-Lab-VCET-2026/ml-lab-2026-johndoe>
- Click your repo link → Clone → Copy HTTPS URL

Step 2: Clone to VS Code — 2 min *

```
git clone https://github.com/ML-Lab-VCET-2026/ml-lab-2026-johndoe.git
cd ml-lab-2026-johndoe
code .
```

VS Code opens with all experiments folders

Step 3: Doing Each Experiment

01	02
Do the experiment	Update readme file
03	04
Commit it	Push to Github

Dr. Sheena N.

GitHub Classroom

PART 3 : FACULTY SIDE - Tracking

Open [classroom.github.com](#) → Your classroom → Assignment

Checking a Student's Work — 30 sec per student *

- Click student repo name **ml-lab-2026-johndoe**
- Click the zip, check it and give marks through issues

Semester End — Download All Records

- Classroom dashboard → Download → Download student repositories
- You get ZIP: **ml-lab-2026-5-experiments.zip** containing 60 folders
- Each folder has 5 PDFs. This is your official record set. Store for NAAC.

Dr. Sheena N.

Thank
you!